

Mutaciones

En lo que respecta a actualizaciones y borrados, es bien sabido que las bases de datos analíticas y transaccionales adoptan **enfoques diferentes** debido a sus filosofías de diseño subyacentes y a los casos de uso a los que van dirigidas. *ClickHouse* es una base de datos orientada a columnas optimizada para **OLAP** de manera que, en la práctica, supone convertir borrados y actualizaciones en inserciones de adición que se procesan de forma asíncrona y/o síncrona (en el momento de la lectura)

Existen dos enfoques fundamentales para actualizar/borrar datos en *ClickHouse*:

- Utilizar **motores de tabla especializados** que gestionan las actualizaciones mediante inserciones
- Utilizar **sentencias declarativas** como `UPDATE ... SET` (**updates ligeros**) o `ALTER TABLE ... UPDATE` (**mutaciones**)

A su vez, dentro de cada enfoque hay varias formas posibles de actualizar/borrar datos, cada una con sus ventajas e inconvenientes. En general, el factor decisivo es siempre el número de datos que intervienen en la operación. Las sentencias declarativas resultan más adecuadas actualizar/borrar un número reducido de filas y/o poco frecuentes



Mutaciones y rendimiento

Aunque sintácticamente las mutaciones pueden parecer similares a las operaciones SQL estándar, su funcionamiento interno es fundamentalmente diferente. En lugar de modificar las filas inmediatamente, las mutaciones en *ClickHouse* son **procesos asíncronos en segundo plano** que reescriben partes completas de datos afectadas por el cambio. Este enfoque es necesario debido al **modelo de almacenamiento inmutable y orientado a columnas** de *ClickHouse* que, mal utilizado, puede provocar un elevado consumo de memoria y I/O

Cuando se emite una mutación, *ClickHouse* programa la creación de nuevas partes mutadas, dejando las partes originales intactas hasta que las nuevas estén listas. Una vez listas, las partes mutadas sustituyen de forma atómica a las originales. Sin embargo, dado que la operación reescribe partes completas, incluso un cambio menor (como la actualización de una sola fila) puede dar lugar a reescrituras a gran escala. Además, las mutaciones siguen un **orden total**: se aplican primero a los datos modificados antes de que se ejecutara la mutación, mientras que los datos más recientes no se ven afectados. No bloquean las inserciones, pero pueden solaparse con otras consultas en curso. Consecuentemente, una consulta `SELECT` que se ejecute durante una mutación puede leer una mezcla de partes mutadas y no mutadas, lo que puede dar lugar a **vistas inconsistentes** de los datos durante la ejecución

Como norma general, es conveniente evitar las mutaciones frecuentes o a gran escala, especialmente en tablas de gran volumen. En su lugar, es recomendable utilizar motores [MergeTree](#) como [ReplacingMergeTree](#) o [CollapsingMergeTree](#) que están específicamente diseñados para gestionar las modificaciones de datos de forma eficiente. Si, pese a todo, fuera imperiosamente necesario es preceptivo monitorizar las mutaciones utilizando la tabla `system.mutations` y, llegado el caso, invocar `KILL MUTATION` si el proceso se bloquea o funciona incorrectamente



Modificaciones de datos en *ClickHouse*

- [Updates en ClickHouse](#)
- [Deletes en ClickHouse](#)